

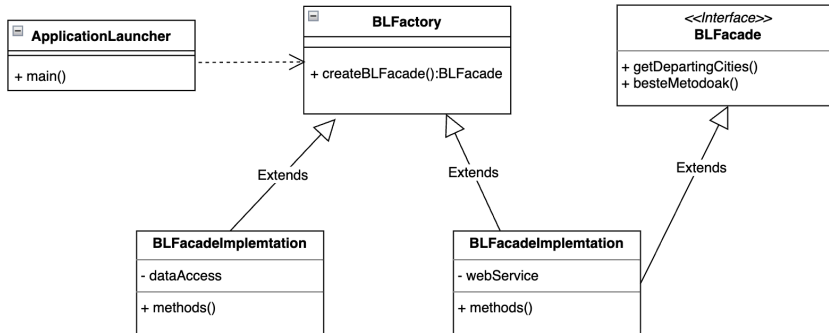
RIDES-EN DISEINU PATROIAK

Eneko Ruiz, Itxaso Urgoitia eta Oihane Pereira

GitHub-eko esteka: <https://github.com/ItxasoUrgoitia/Rides24/tree/master>

1. FACTORY METHOD PATROIA

a) UML Diagrama Hedatua



b) Aldaketa nagusiak kodean

Aurretik:

Aurreko kodean, `ApplicationLauncher`-eko `main()` metodoak zuzenean erabakitzen zuen zein motatako konexioa erabili. Kodea nahasia zegoen eta konfigurazioa irakurtzeaz gain, web zerbitzuaren konexioa zuzenean kudeatzen genuen.

```
package gui;
import java.awt.Color;
import java.net.URL;
import java.util.Locale;
import javax.swing.UIManager;
import javax.xml.namespace.QName;
import javax.xml.ws.Service;
import configuration.ConfigXML;
import dataAccess.DataAccess;
import businessLogic.BLFacade;
import businessLogic.BLFacadeImplementation;
public class ApplicationLauncher {

    public static void main(String[] args) {
        ConfigXML c=ConfigXML.getInstance();
        System.out.println(c.getLocale());
        Locale.setDefault(new Locale(c.getLocale()));
        System.out.println("Locale: "+Locale.getDefault());
        MainGUI a=new MainGUI();
        a.setVisible(true);
        try {
            BLFacade appFacadeInterface;
            UIManager.setLookAndFeel("javax.swing.plaf.metal.MetalLookAndFeel");
            if (c.isBusinessLogicLocal()) {
                DataAccess da= new DataAccess();
                appFacadeInterface=new BLFacadeImplementation(da);
            }
            else { //If remote
                String serviceName= "http://"+c.getBusinessLogicNode() +":"+c.getBusinessLogicPort()+"/ws/"+c.getBusinessLogicName()+"?wsdl";
                URL url = new URL(serviceName);
```

```

        QName qname = new QName("http://businessLogic/", "BLFacadeImplementationService");
        Service service = Service.create(url, qname);
        appFacadeInterface = service.getPort(BLFacade.class);
    }
    MainGUI.setBusinessLogic(appFacadeInterface);
} catch (Exception e) {
    a.jLabelSelectOption.setText("Error: " + e.toString());
    a.jLabelSelectOption.setForeground(Color.RED);
    System.out.println("Error in ApplicationLauncher: " + e.toString());
}
}
}

```

Ondoren:

BLFactory klase berri bat sortu dugu. Klase honen eginkizuna da objektuen sorrera prozesua zentralizatzea. Factoryak konfigurazioa irakurri eta dagokion inplementazioa sortzen du, kodea sinplifikatuz. ApplicationLauncher-en kodea asko garbitu da, factoryari deia egitera murriztuz.

```

package businessLogic;
import java.net.URL;
import javax.xml.namespace.QName;
import javax.xml.ws.Service;
import configuration.ConfigXML;
import dataAccess.DataAccess;
public class BLFactory {
    public static BLFacade createBLFacade() throws Exception {
        ConfigXML c = ConfigXML.getInstance();
        BLFacade appFacadeInterface;
        if (c.isBusinessLogicLocal()) {
            DataAccess da = new DataAccess();
            appFacadeInterface = new BLFacadeImplementation(da);
        } else {
            String serviceName = "http://" + c.getBusinessLogicNode() + ":" +
                c.getBusinessLogicPort() + "/ws/" +
                c.getBusinessLogicName() + "?wsdl";
            URL url = new URL(serviceName);
            QName qname = new QName("http://businessLogic/", "BLFacadeImplementationService");
            Service service = Service.create(url, qname);
            appFacadeInterface = service.getPort(BLFacade.class);
        }
        return appFacadeInterface;
    }

    public BLFacade getBusinessLogicFactory(boolean isLocal) throws Exception {
        if (isLocal) {
            return new BLFacadeImplementation(new DataAccess());
        } else {
            URL url = new URL("http://localhost:9999/ws?wsdl");
            QName qname = new QName("http://businessLogic/", "BLFacadeImplementationService");
            Service service = Service.create(url, qname);
            return service.getPort(BLFacade.class);
        }
    }
}

```

```

package gui;
import java.awt.Color;
import java.net.URL;
import java.util.Locale;
import javax.swing.UIManager;
import javax.xml.namespace.QName;
import javax.xml.ws.Service;
import configuration.ConfigXML;
import dataAccess.DataAccess;
import businessLogic.BLFacade;
import businessLogic.BLFacadeImplementation;
import businessLogic.BLFactory;
public class ApplicationLauncher {

    public static void main(String[] args) {
        ConfigXML c=ConfigXML.getInstance();
        System.out.println(c.getLocale());
        Locale.setDefault(new Locale(c.getLocale()));
        System.out.println("Locale: "+Locale.getDefault());
        MainGUI a=new MainGUI();
        a.setVisible(true);
        try {

            UIManager.setLookAndFeel("javax.swing.plaf.metal.MetalLookAndFeel");
            BLFacade appFacadeInterface = BLFactory.createBLFacade();
            MainGUI.setBussinessLogic(appFacadeInterface);
            MainGUI.setBussinessLogic(appFacadeInterface);
        }catch (Exception e) {
            a.jLabelSelectOption.setText("Error: "+e.toString());
            a.jLabelSelectOption.setForeground(Color.RED);
            System.out.println("Error in ApplicationLauncher: "+e.toString());
        }
    }
}

```

c) Azalpena

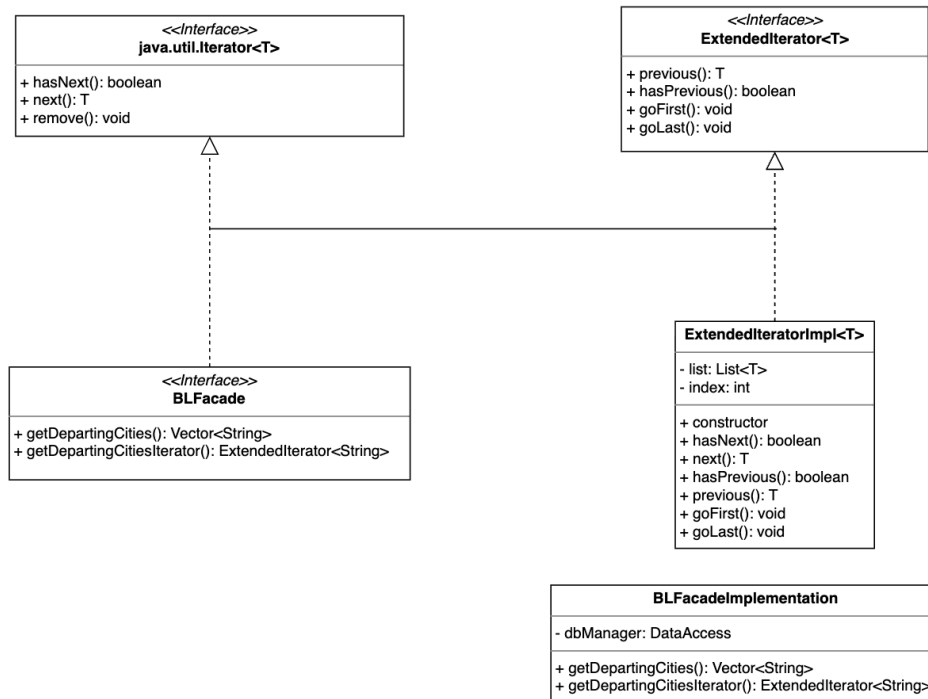
Factory Method patroia ren abantaila nagusia objektuen sorrera logika kodea guztitik banatzea da. Gure kasuan, BLFactory klaseak jakiten du nola sortu konexio mota bakoitza, eta bezero kodeak ez du jakin behar horren xehetasunik.

Etorkizunean konexio mota berri bat gehitu behar badugu, factorya bakarrik aldatu behar dugu, aplikazioaren gainontzeko zatia ukitu gabe. Gainera, testaketa ere errazagoa da, factoryak mock objektuak sor ditzakeelako testak egiteko.

Kodearen irakurgarritasuna ere hobetu da, sorrera logika konplexua factory batean isolatu dugu eta bezeroak interfaze bakarra erabiltzeko aukera du.

2. ITERATOR PATROIA (Extended iterator)

a) UML Diagrama Hedatua



b) Aldaketa nagusiak kodean

Aurretik:

Aurreko kodean, `getDepartingCities()` metodoak hiri guztiak batera itzultzen zituen `Vector` batean:

```

public Vector<String> getDepartingCities() {
    dbManager.open();
    Vector<String> cities = dbManager.getDepartCities();
    dbManager.close();
    return cities;
}
  
```

Ondoren:

`ExtendedIterator` interfaze berria sortu dugu, Java-ren `Iterator` estandarra hedatzen duena. Interfaze honek aukera ematen du hiriak modu flexibleagoan ikusteko.

```

package businessLogic;
import java.util.Iterator;
public interface ExtendedIterator<Object> extends Iterator<Object> {
    //return the actual element and go to the previous
    public Object previous();
    //true if there is a previous element
    public boolean hasPrevious();
    //It is placed in the first element
  
```

```

    public void goFirst();
    //It is placed in the last element
    public void goLast();
}

```

Sortu dugun ExtendedIteratorImpl klaseak interfaze honen implementazioa egiten du:

```

package businessLogic;
import java.util.List;
import java.util.NoSuchElementException;
public class ExtendedIteratorImpl<T> implements ExtendedIterator<T> {
    private List<T> list;
    private int index;
    public ExtendedIteratorImpl(List<T> list) {
        this.list = list;
        goFirst();
    }
    @Override
    public boolean hasNext() {
        return index < list.size();
    }
    @Override
    public T next() {
        if (!hasNext()) throw new NoSuchElementException();
        return list.get(index++);
    }
    @Override
    public boolean hasPrevious() {
        return index > 0;
    }
    @Override
    public T previous() {
        if (!hasPrevious()) throw new NoSuchElementException();
        index--;
        return list.get(index);
    }
    @Override
    public void goFirst() {
        index = 0;
    }
    @Override
    public void goLast() {
        index = list.size();
    }
}

```

Eta BLFacadeImplementation-en metodo berria gehitu dugu:

```

@WebMethod
public ExtendedIterator<String> getDepartingCitiesIterator(){
    dbManager.open();
    List<String> cities = dbManager.getDepartCities(); // obtenemos la lista
    dbManager.close();
    return new ExtendedIteratorImpl<>(cities); // devolvemos un iterador concreto
}

```

BLFacaden hurrengoa gehitu dugu:

```

public ExtendedIterator<String> getDepartingCitiesIterator();

```

Eta ApplicationLauncher-en hau gehitu dugu (MainGUI.setBussinessLogic(blFacade); ondoren):

```
ExtendedIterator<String> i = blFacade.getDepartingCitiesIterator();
    String c;
    System.out.println("_____");
    System.out.println("FROM    LAST    TO    FIRST");
    i.goLast(); // Go to last element
    while (i.hasPrevious()) {
        c = i.previous();
        System.out.println(c);
    }
    System.out.println();
    System.out.println("_____");
    System.out.println("FROM    FIRST    TO    LAST");
    System.out.print("RIDES:" + blFacade.getAllRides());
    i.goFirst(); // Go to first element
    while (i.hasNext()) {
        c = i.next();
        System.out.println(c);
    }
```

c) Azalpena

Iterator patroiaaren abantaila nagusia bilduma baten elementuak ikusteko modu estandarizatua eta malgua eskaintzea da. Gure ExtendedIterator-ek ahalbidetzen du bilduma bat bi noranzkoetan korritzea, eta edozein posiziotatik hastea.

Honen abantaila da bildumaren barne egitura ezkutatzen dela. Bezeroak ez du jakin behar nola dauden bildumako elementuak antolatuta, soilik iterazio interfazea erabiltzen du. Horrek ahalbidetzen du bildumaren inplementazioa aldatzea bezero kodea ukitu gabe.

Gure kasuan, hiriak modu desberdinetan bistara ditzakegu: ordena normalean, alderantzizko ordenan, edo zati espezifikotatik hasita. Hau oso erabilgarria da interfaze grafikoetan edo datuak modu desberdinetan erakusteko.

d) Exekuzioaren irudia

main/java/gui/ApplicationLauncher.java - Eclipse IDE

Project Run Window Help Copilot

BLFacadeImp... BLFacade.java Application... ExtendedIte... ExtendedIte... »13

```
53 BLFacade blFacade = new BLFactory().getBusinessLogicFactory(isLocal);
54
55
56 MainGUI.setBusinessLogic(blFacade);
57
58
59 MainGUI.setBusinessLogic(blFacade);
60
61 ExtendedIterator<String> i = blFacade.getDepartingCitiesIterator();
62 String c;
63 System.out.println("_____");
64 System.out.println("FROM LAST TO FIRST");
65 i.goLast(); // Go to last element
```

Sonar... Problem... Chat @ Javadoc Search Covera... Declara... Console × Terminal Git Sta...

ApplicationLauncher (2) [Java Application] C:\Users\itxas\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_23.0.1.v20241024-17

DataAccess opened => isDatabaseLocal: true
LORTUTAKO HIRIAKKKKKK: [Barcelona, Bilbo, Donostia, Gasteiz, Iruña, Madrid]
DataAccess closed

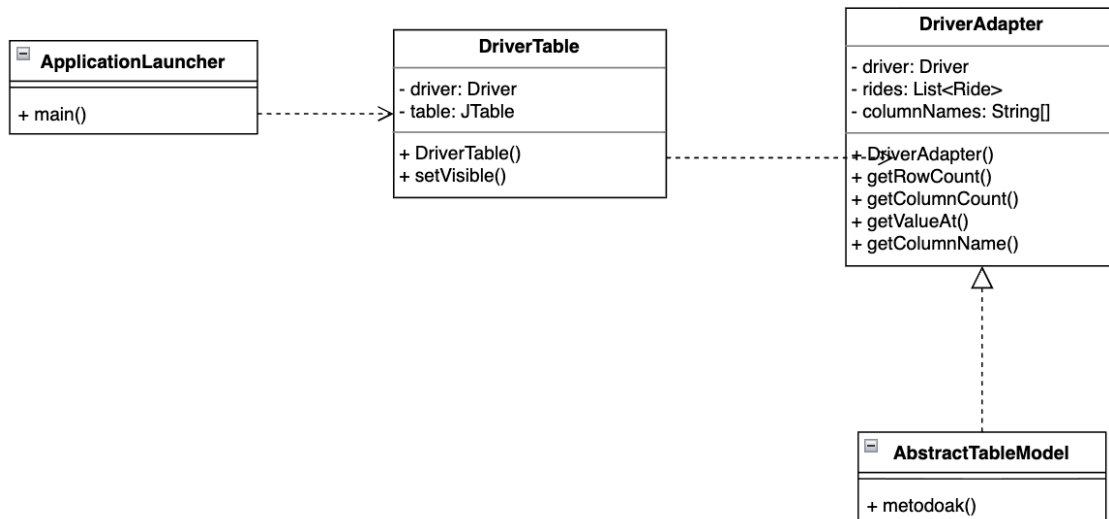
FROM	LAST	TO	FIRST
Madrid			
Iruña			
Gasteiz			
Donostia			
Bilbo			
Barcelona			

FROM	FIRST	TO	LAST
Barcelona			
Bilbo			
Donostia			
Gasteiz			
Iruña			
Madrid			

irroia25\git
\git
git
4Objectdb2\
1-JUnit\git
terns\git

3. ADAPTER PATROIA

a) UML Diagrama Hedatua



b) Aldaketa nagusiak kodean

Aurretik:

Aurreko kodean, Driver klasearen datuak eta JTable osagaia ez ziren zuzenean bateragarriak. Driver-ek negozio logikaren interfazea erabiltzen du, eta JTable-k Swing-en TableModel interfazea espero du.

Ondoren:

DriverAdapter klase berria sortu dugu, AbstractTableModel heredatzen duena. Adapter honen eginkizuna da Driver objektuaren datuak JTable-rentzat egokiak diren formatura moldatzea.

```
package businessLogic;
import java.util.List;
import javax.swing.table.AbstractTableModel;
import domain.Driver;
import domain.Ride;
public class DriverAdapter extends AbstractTableModel{
    private Driver driver;
    private String[] columnNames = { "From", "To", "Price" };
    private List<Ride> rides;
    public DriverAdapter(Driver driver) {
        this.driver = driver;
        this.rides = driver.getRides(); // suponiendo que existe getRides()
    }

    @Override
    public int getRowCount() {
        return rides != null ? rides.size() : 0;
    }

    @Override
    public int getColumnCount() {
```

```

        return columnNames.length;
    }
    @Override
    public Object getValueAt(int rowIndex, int columnIndex) {
        Ride ride = rides.get(rowIndex);
        switch (columnIndex) {
            case 0: return ride.getFrom();
            case 1: return ride.getTo();
            case 2: return ride.getPrice();
            default: return null;
        }
    }
    @Override
    public String getColumnName(int column) {
        return columnNames[column];
    }
}

```

c) Azalpena

Adapter patroiairen abantaila nagusia interfaze bateraezinak dituzten klaseak elkarrekin lan egitera behartzea da. Gure kasuan, DriverAdapter-ek bi mundu lotzen ditu: gure domain objektuak eta Swing interfaze grafikoak.

Honek kodea asko sinplifikatzen du. DriverTable-k ez du jakin behar nola antolatuta dauden Driver-en barne datuak, soilik adapterra erabiltzen du taula bat sortzeko. Datuen transformazio logika adapterren isolatuta dago, kodea mantentzeko errazagoa eginez.

Gainera, beste taula mota batzuk sortu nahi baditugu, adapter berriak sor ditzakegu eredu berari jarraituz.

d) Exekuzioaren irudi

